



End-to-End DevSecOps CI/CD Pipeline for Zomato Web Application

An end-to-end **DevSecOps CI/CD pipeline** for deploying a Zomato-style Node.js web application on AWS using **Terraform, Jenkins, SonarQube, npm audit, Trivy, Docker, and Docker Hub**.

The project demonstrates Infrastructure as Code, continuous integration, static code analysis, dependency security scanning, container vulnerability scanning, automated Docker image publishing, and application deployment on an AWS EC2 instance.



Project Overview

This project implements an automated DevSecOps workflow where application source code is maintained in GitHub and automatically processed through a Jenkins CI/CD pipeline.

Terraform is used to provision the AWS infrastructure, while Jenkins automates the application build and security validation process.

The pipeline performs:

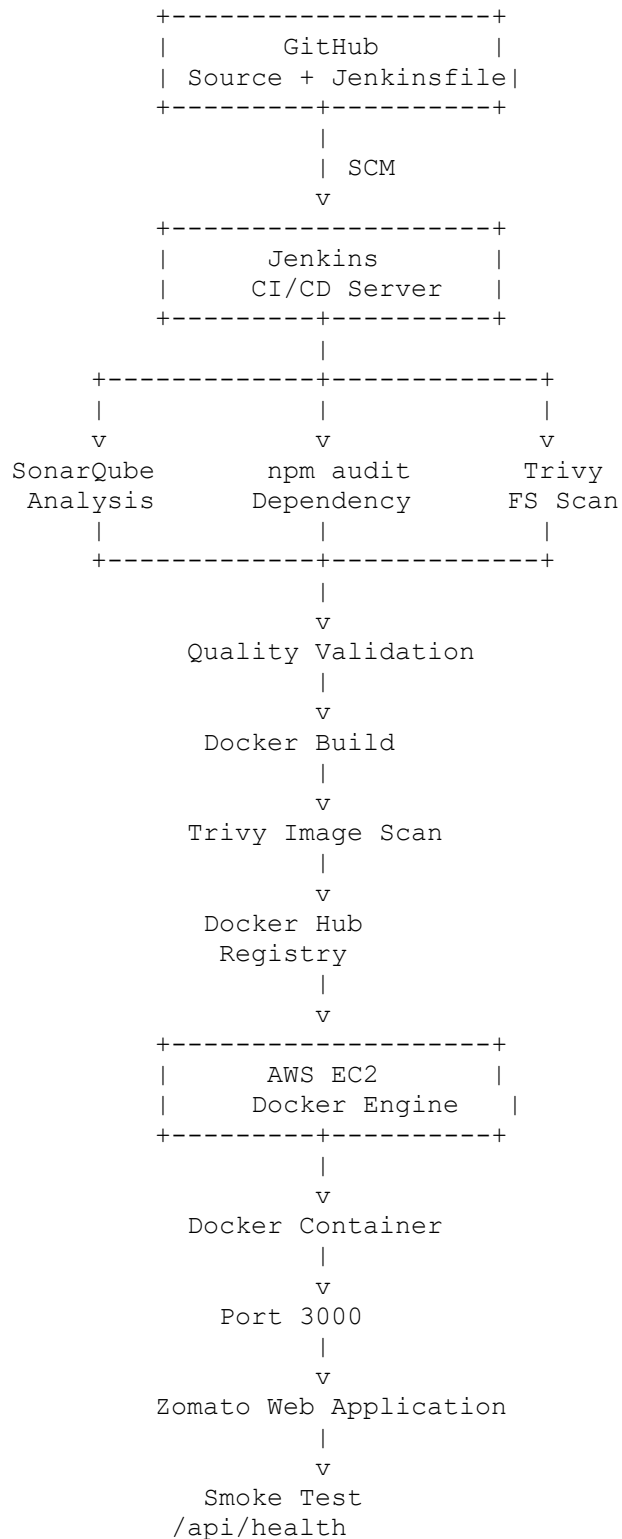
- Source code checkout
- Dependency installation
- SonarQube static code analysis
- SonarQube Quality Gate validation
- NPM dependency security audit
- Trivy filesystem security scanning
- Docker image creation
- Trivy Docker image scanning
- Docker Hub image publishing
- Container deployment
- Application health verification

The final Zomato-style web application is deployed inside a Docker container on AWS EC2 and is accessible through port 3000.



Architecture

```
Developer
|
| Git Push
v
```



AWS Infrastructure

The application is deployed on AWS using Terraform.

AWS Region

ap-south-1

Infrastructure Components

- Amazon EC2
- VPC networking
- Security Group
- EBS storage
- Jenkins
- Docker Engine
- SonarQube
- Trivy

Application Ports

Port	Purpose
------	---------

22	SSH
----	-----

8080	Jenkins
------	---------

9000	SonarQube
------	-----------

3000	Zomato application
------	--------------------



Technology Stack

Technology	Purpose
------------	---------

AWS EC2	Cloud compute infrastructure
---------	------------------------------

Terraform	Infrastructure as Code
-----------	------------------------

GitHub	Source code management
--------	------------------------

Jenkins	CI/CD automation
---------	------------------

SonarQube	Static code analysis
-----------	----------------------

npm audit	Dependency vulnerability analysis
-----------	-----------------------------------

Trivy	Filesystem and container vulnerability scanning
-------	---

Docker	Application containerization
--------	------------------------------

Docker Hub	Container image registry
------------	--------------------------

Node.js	Application runtime
---------	---------------------

Express.js	Backend framework
------------	-------------------

HTML5	Application UI
-------	----------------

CSS3	Styling
------	---------

JavaScript	Frontend functionality
------------	------------------------



Project Structure

```
zomato-devops-project/
├── app/
│   ├── public/
│   │   ├── css/
│   │   │   └── style.css
│   │   └── js/
│   │       ├── app.js
│   │       └── index.html
│   ├── package.json
│   ├── package-lock.json
│   └── server.js
├── terraform/
│   ├── install.sh
│   ├── main.tf
│   ├── outputs.tf
│   ├── terraform.tfvars.example
│   ├── variables.tf
│   └── versions.tf
├── .gitignore
├── Dockerfile
├── Jenkinsfile
└── README.md
```



CI/CD Pipeline

The Jenkins pipeline contains the following stages:

#	Stage	Purpose
1	Clean Workspace	Removes previous build files
2	Checkout Source Code	Retrieves source from GitHub
3	Install Dependencies	Installs Node.js dependencies
4	SonarQube Analysis	Performs static code analysis
5	Quality Gate	Validates SonarQube results
6	Security Dependency Audit	Runs <code>npm audit</code>
7	Trivy Filesystem Scan	Scans source files for vulnerabilities/secrets
8	Docker Build	Builds the application image
9	Trivy Image Scan	Scans the Docker image
10	Docker Hub Push	Publishes the Docker image
11	Deploy Container	Runs the application container
12	Smoke Test	Validates application health

□ Clean Workspace

The first stage cleans the Jenkins workspace before starting a new build.

```
cleanWs()
```

Purpose

This prevents files from previous builds from affecting the current pipeline execution.

❑Checkout Source Code

Jenkins retrieves the latest application source code from GitHub.

```
GitHub  
↓  
Jenkins
```

This ensures that the pipeline always works with the version committed to the repository.

❑Install Dependencies

The application dependencies are installed using NPM.

```
npm install --production
```

The application uses Node.js and Express.js.

❑SonarQube Analysis

SonarQube performs static analysis of the application source code.

It helps identify:

- Bugs
- Code smells
- Security issues
- Maintainability problems
- Vulnerabilities

The analysis is integrated directly into the Jenkins pipeline.

5 SonarQube Quality Gate

After the SonarQube analysis, Jenkins waits for the SonarQube Quality Gate result.

```
Jenkins
  |
  v
SonarQube Analysis
  |
  v
Quality Gate
  |
+----- PASSED -----> Continue
  |
+----- FAILED -----> Pipeline validation fails
```

A SonarQube webhook is configured to communicate the Quality Gate result back to Jenkins.

Webhook endpoint:

```
http://<EC2_PUBLIC_IP>:8080/sonarqube-webhook/
```

6 NPM Dependency Audit

The pipeline performs dependency security analysis using:

```
npm audit
```

This checks third-party NPM packages for known vulnerabilities.

7 Trivy Filesystem Scan

Trivy is used to scan the project filesystem.

Example:

```
trivy fs .
```

The scan helps identify vulnerabilities and exposed secrets within the project source and dependencies.

8 Docker Image Build

The application is packaged into a Docker image.

Example:

```
docker build -t zomato-devops-app .
```

Docker provides a consistent environment for running the application.

Trivy Docker Image Scan

After building the Docker image, Trivy scans the image for vulnerabilities.

Example:

```
trivy image zomato-devops-app
```

This provides an additional security layer before the image is published.

10 Docker Hub Push

After successful validation, the Docker image is pushed to Docker Hub.

Example:

```
<DOCKER_USERNAME>/zomato-devops-app:latest
```

Docker Hub acts as the container image registry.

Container Deployment

The Docker image is deployed on the AWS EC2 instance.

The application container exposes:

```
3000:3000
```

The application can then be accessed using:

```
http://<EC2_PUBLIC_IP>:3000
```

Smoke Test

The final pipeline stage verifies that the application is running successfully.

Health endpoint:

`/api/health`

Example:

```
curl http://localhost:3000/api/health
```

Expected result is a successful HTTP response from the application.



Docker

The application is containerized using Docker.

Docker workflow

```
Application Source
  |
  v
Dockerfile
  |
  v
Docker Image
  |
  v
Docker Hub
  |
  v
Docker Container
```

Useful Docker commands

```
docker build -t zomato-devops-app .
docker images
docker ps
docker logs <container-name>
docker stop <container-name>
docker rm <container-name>
```



DevSecOps Security

Security is integrated throughout the CI/CD pipeline instead of being performed only after deployment.

The project implements multiple security checks:

```
Source Code
|
+----> SonarQube
|
+----> npm audit
|
+----> Trivy Filesystem
|
+----> Trivy Docker Image
|
v
Docker Deployment
```

Security tools

SonarQube

Used for static code analysis.

npm audit

Used for identifying vulnerable NPM dependencies.

Trivy

Used for:

- Filesystem vulnerability scanning
- Secret detection
- Container image vulnerability scanning



Deployment Environment

AWS Region

ap-south-1

Jenkins

http://<EC2_PUBLIC_IP>:8080

SonarQube

http://<EC2_PUBLIC_IP>:9000

Application

`http://<EC2_PUBLIC_IP>:3000`

Health Check

`http://<EC2_PUBLIC_IP>:3000/api/health`

Replace `<EC2_PUBLIC_IP>` with the public IP address of the EC2 instance. Do not commit environment-specific IP addresses if they are not intended to be permanent.



Deployment Process

Step 1 — Configure AWS CLI

Configure AWS credentials locally:

```
aws configure
```

Verify the configuration:

```
aws sts get-caller-identity
```

Step 2 — Initialize Terraform

Navigate to the Terraform directory:

```
cd terraform
```

Initialize Terraform:

```
terraform init
```

Step 3 — Validate Terraform

```
terraform validate
```

Expected result:

```
Success! The configuration is valid.
```

Step 4 — Review Infrastructure

```
terraform plan
```

Review the resources that Terraform intends to create.

Step 5 — Provision Infrastructure

```
terraform apply
```

Confirm the deployment when prompted.

After successful execution, Terraform provisions the required AWS infrastructure.



Server Configuration

After provisioning the EC2 instance, connect through SSH:

```
ssh -i "path/to/zomato-devops-key.pem" ubuntu@<EC2_PUBLIC_IP>
```

Verify Docker:

```
docker --version
```

Verify Java:

```
java --version
```

Verify Jenkins:

```
sudo systemctl status jenkins
```

Verify Docker:

```
sudo systemctl status docker
```

Verify Trivy:

```
trivy --version
```



Jenkins Configuration

Jenkins is available on:

```
http://<EC2_PUBLIC_IP>:8080
```

Retrieve the initial administrator password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

After unlocking Jenkins, configure the required plugins and tools.

Jenkins tools

- JDK 17
- Node.js 18
- SonarQube Scanner
- OWASP Dependency Check
- Docker-related plugins



Jenkins Credentials

Credentials are stored using Jenkins Credentials rather than hard-coded in source code.

Examples include:

```
sonar-token  
docker
```

Credentials should never be committed to GitHub.

Never store:

```
AWS Access Keys  
AWS Secret Keys  
Docker Hub passwords  
SonarQube tokens  
SSH private keys
```

inside the Git repository.



SonarQube

SonarQube is available at:

```
http://<EC2_PUBLIC_IP>:9000
```

The Jenkins pipeline sends application source code to SonarQube for analysis.

The Quality Gate then determines whether the pipeline can continue.



Docker Hub

The Docker image is published to the project's Docker Hub repository.

Example:

```
<DOCKER_USERNAME>/zomato-devops-app
```

The image can then be pulled by the deployment environment.



Testing

The project was tested at multiple levels.

Application test

```
curl http://localhost:3000/api/health
```

Docker container

```
docker ps
```

Container logs

```
docker logs <container-name>
```

Application browser test

```
http://<EC2_PUBLIC_IP>:3000
```

Jenkins

Verify that all pipeline stages complete successfully.

SonarQube

Verify that the project analysis and Quality Gate complete successfully.

Docker Hub

Verify that the Docker image has been published successfully.



Project Screenshots

Recommended screenshots for this project:

AWS

- EC2 instance running
- EC2 public IP
- Security Group
- Terraform execution

Terraform

- `terraform init`
- `terraform validate`
- `terraform plan`
- `terraform apply`

Jenkins

- Jenkins dashboard
- Jenkins credentials configuration
- Jenkins tools configuration
- Pipeline execution
- Successful pipeline
- Individual pipeline stages

SonarQube

- SonarQube project
- Code analysis
- Quality Gate
- Analysis results

Security

- npm audit result
- Trivy filesystem scan
- Trivy Docker image scan
- OWASP/security results if configured

Docker

- Docker image
- Running container
- Docker Hub repository

Application

- Zomato application homepage
 - Final deployed application
 - `/api/health` response
-



Security Best Practices

This project follows several security practices:

- Credentials are managed through Jenkins Credentials.
 - Secrets are not stored in source code.
 - Terraform variables are used for environment-specific configuration.
 - `.gitignore` prevents sensitive files from being committed.
 - SonarQube provides static code analysis.
 - `npm audit` checks third-party dependencies.
 - Trivy scans the project filesystem.
 - Trivy scans the Docker image before deployment.
 - Docker images are stored in Docker Hub.
 - AWS infrastructure is managed through Terraform.
-



Cleanup

AWS resources should be destroyed when the project is no longer required to avoid unnecessary charges.

Navigate to the Terraform directory:

```
cd terraform
```

Run:

```
terraform destroy
```

Confirm the operation.

After destruction, verify in the AWS Console that the Terraform-created resources have been removed.

Check:

- EC2
- EBS

- Security Groups
- Any other resources created by Terraform



DevSecOps Workflow

The completed project demonstrates the following workflow:

```
CODE
|
v
GitHub
|
v
Jenkins
|
v
Install Dependencies
|
v
SonarQube SAST
|
v
Quality Gate
|
v
npm audit
|
v
Trivy FS Scan
|
v
Docker Build
|
v
Trivy Image Scan
|
v
Docker Hub
|
v
AWS EC2 Container
|
v
Smoke Testing
|
v
LIVE APPLICATION
```



Project Objectives Achieved

The project successfully demonstrates:

-  AWS cloud deployment
-  Infrastructure as Code with Terraform
-  GitHub source control
-  Jenkins CI/CD automation
-  SonarQube static code analysis
-  Quality Gate integration
-  NPM dependency security audit
-  Trivy filesystem scanning
-  Trivy container image scanning
-  Docker containerization
-  Docker Hub image publishing
-  Automated container deployment
-  Application health checks
-  DevSecOps security integration



Key Learnings

Through this project, the following DevOps concepts were implemented:

Infrastructure as Code

Terraform was used to automate AWS infrastructure provisioning instead of creating resources manually.

Continuous Integration

Jenkins automatically retrieves and validates application source code from GitHub.

Continuous Delivery

The pipeline automatically builds, scans, publishes, and deploys the Docker image.

Shift-Left Security

Security checks are performed during the CI/CD process rather than waiting until after deployment.

Containerization

Docker packages the application and its runtime dependencies into a portable container.

Automated Security

SonarQube, npm audit, and Trivy provide multiple layers of application and container security validation.



Future Improvements

Possible improvements for a production-grade implementation include:

- Implement HTTPS using a domain and TLS certificate.
 - Use AWS IAM roles instead of long-lived access keys.
 - Move secrets to AWS Secrets Manager or Parameter Store.
 - Use a private Docker registry.
 - Use Amazon ECR instead of Docker Hub.
 - Introduce Terraform remote state using S3 and state locking.
 - Deploy using ECS/EKS instead of a single EC2 instance.
 - Add automated GitHub webhooks.
 - Add monitoring using Amazon CloudWatch.
 - Add centralized logging.
 - Implement blue-green or rolling deployments.
 - Add automated rollback.
 - Introduce infrastructure monitoring and alerting.
 - Add automated unit and integration tests.
 - Add dependency update automation.
-



Conclusion

This project demonstrates a complete **end-to-end DevSecOps CI/CD implementation** for a Zomato-style web application.

Terraform automates AWS infrastructure provisioning, Jenkins orchestrates the CI/CD workflow, SonarQube performs static code analysis, npm audit and Trivy provide security validation, Docker containerizes the application, Docker Hub stores the image, and the final container is deployed on AWS EC2.

The project demonstrates how development, infrastructure, CI/CD, containerization, security, and deployment can be integrated into a single automated workflow.



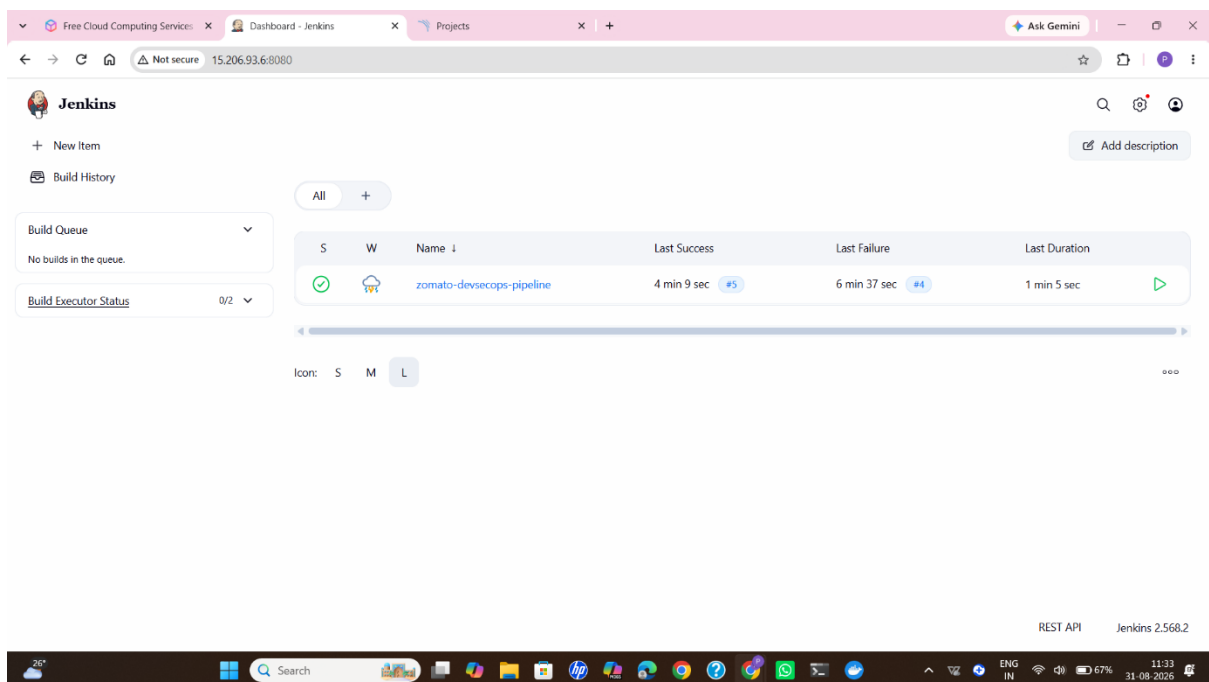
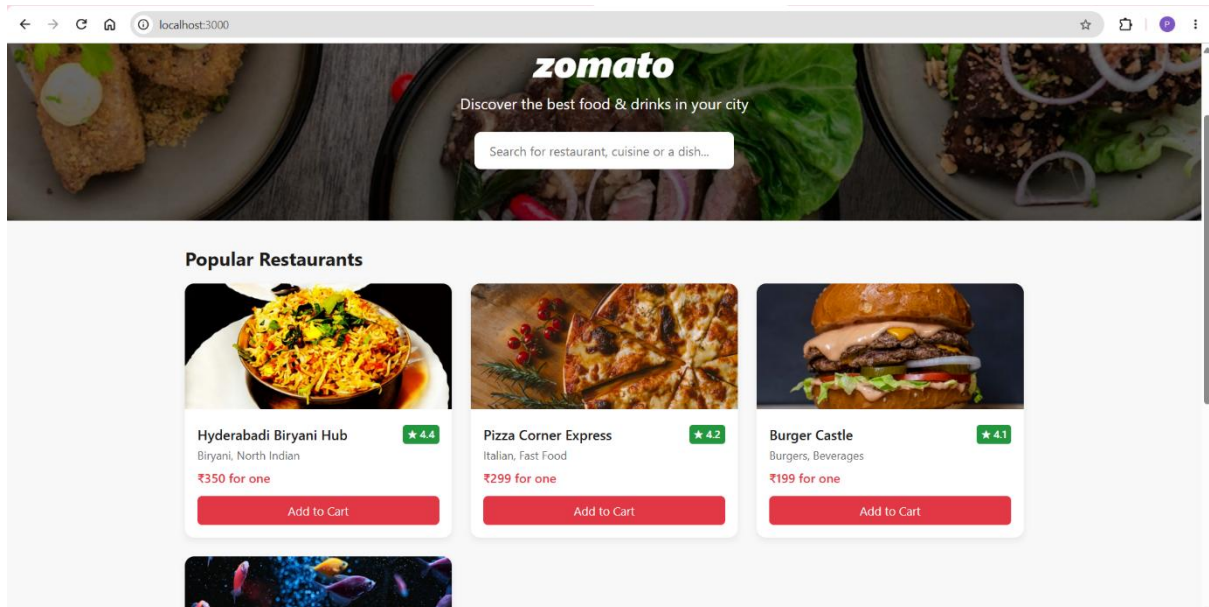
Author

Pavan Kumar

GitHub: <YOUR_GITHUB_PROFILE>

Docker Hub: <YOUR_DOCKERHUB_PROFILE>

Screenshots



Free Cloud Computing Services x zomato-devseccps-pipeline - J... x Projects x + Ask Gemini

15.206.93.6:9000/projects

You're running a version of SonarQube that is no longer active. Please upgrade to an active version immediately. [Learn More](#)

sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration

Search for projects...

Filters

Quality Gate

Passed 1

Failed 0

Reliability (Bugs)

A rating 1

B rating 0

C rating 0

D rating 0

E rating 0

Security (Vulnerabilities)

A rating 1

B rating 0

C rating 0

D rating 0

E rating 0

Security Review (Security Hotspots)

A rating 0

B rating 0

C rating 0

D rating 0

E rating 0

Search by project name or key

1 project(s)

Perspective: Overall Status Sort by: Name

Create Project

Zomato DevOps App Passed Last analysis: 34 minutes ago

Bugs 0 A Vulnerabilities 0 A Hotspots Reviewed 0.0% E Code Smells 0 A Coverage 0.0% Duplications 0.0% Lines 266 XS CSS, Java...

1 of 1 shown

Embedded database should be used for evaluation purposes only

The embedded database will not scale, it will not support upgrading to newer versions of SonarQube, and there is no support for migrating your data out of it into a different database engine.

SonarQube™ technology is powered by SonarSource SA

Community Edition - v9.9.8 (build 100196) NO LONGER ACTIVE - LGPL v3 - Community - Documentation - Plugins - Web API

Zomato DevOps Project Setup x Zomato - Discover the Best Food x + Ask Gemini

15.206.93.6:3000

Not secure

zomato

Add restaurant Log in Sign up

Discover the best food & drinks in your city

Search for restaurant, cuisine or a dish...

Popular Restaurants

Hyderabadi Biryani Hub 4.4 Biryani, North Indian ₹350 for one

Pizza Corner Express 4.2 Italian, Fast Food ₹299 for one

Burger Castle 4.1 Burgers, Beverages ₹199 for one

EC2 > Key pairs

Instance Types Launch Templates Spot Requests Savings Plans Reserved Instances Dedicated Hosts Capacity Reservations Capacity Manager New

Images AMIs AMI Catalog

Elastic Block Store

Key pairs (1/1) info

Find Key Pair by attribute or tag

Actions Create key pair

Name	Type	Created	Fingerprint	ID
zomato-devops-key	rsa	2026/08/31 09:54 GMT+5:30	cax5:2c:9e:78:36:8d:5c:43:89:3d:b8...	key-02b8e4052e08d1a1f

github login - Search x pavankumarkn-gi/zomato-devops x docker hub login - Search x pavankumarkn/zomato-devops-app x +

https://hub.docker.com/repository/docker/pavankumarkn/zomato-devops-app/general

New Insights on the state of AI agents from 800+ builders and leaders. Download your copy →

hub Explore My Hub Search Docker Hub CtrlK

pavankumarkn Your personal account

Repositories / zomato-devops-app / General

pavankumarkn/zomato-devops-app Last pushed 38 minutes ago · ☆0 · ↓26

Add a description Add a category

Docker commands To push a new tag to this repository: docker push pavankumarkn/zomato-devops-app:tagname

Using 0 of 1 private repositories.

Public view

General Tags Image Management Collaborators Webhooks Settings

Tags This repository contains 1 tag(s).

Tag	OS	Type	Pulled	Pushed
latest	linux	Image	less than 1 day	38 minutes

See all

DOCKER SCOUT INACTIVE Activate

buildcloud Build with Docker Build Cloud

Accelerate image build times with access to cloud-based builders and shared cache.

Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.

Get faster builds through shared caching across your team, native multi-platform support, and encrypted data transfer - all without managing infrastructure.

Upgrade subscription

https://hub.docker.com/repository/docker/pavankumarkn/zomato-devops-app/image-management

pavankumarkn-gi / zomato-devops-terraform-jenkins

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

zomato-devops-terraform-jenkins Public

Pin Watch 0 Fork 0 Star 0

main 1 Branch 0 Tags

Go to file Add file Code

pavankumarkn-gi Initial commit: Zomato web application and Docker configuration 07e4782 · 1 hour ago 1 Commit

app	Initial commit: Zomato web application and Docker configura...	1 hour ago
.dockerignore	Initial commit: Zomato web application and Docker configura...	1 hour ago
.gitignore	Initial commit: Zomato web application and Docker configura...	1 hour ago
Dockerfile	Initial commit: Zomato web application and Docker configura...	1 hour ago
sonar-project.properties	Initial commit: Zomato web application and Docker configura...	1 hour ago

README

Add a README

Help people interested in this repository understand your project.

Add README

About devops

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors 1

pavankumarkn-gi


```
Last login: Mon Aug 31 05:26:50 2026 from 152.57.119.145
ubuntu@ip-172-31-46-21:~$ # Check running containers (SonarQube & Zomato App)
docker ps

# Check tool versions
java -version
trivy --version
CONTAINER ID   IMAGE                                     COMMAND                  CREATED        STATUS        PORTS
e52821cbd5ce   pavankumarkn/zomato-devops-app:latest   "docker-entrypoint.s..." 43 minutes ago Up 43 minutes 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
zomato-app     sonarqube:lts-community                  "/opt/sonarqube/dock..." 2 hours ago    Up 2 hours    0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp
sonar-server
openjdk version "21.0.12" 2026-07-21
OpenJDK Runtime Environment (build 21.0.12+8-1-22.04-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.12+8-1-22.04-Ubuntu, mixed mode, sharing)
Version: 0.74.0
ubuntu@ip-172-31-46-21:~$ |
```

```
{"status": "UP", "timestamp": "2026-08-31T06:16:41.959Z"}
ubuntu@ip-172-31-46-21:~$ cat /var/lib/jenkins/workspace/zomato-devsecops-pipeline/trivy-fs-report.txt | head -n 25
cat /var/lib/jenkins/workspace/zomato-devsecops-pipeline/trivy-fs-report.txt | head -n 25

Report Summary



| Target                | Type | Vulnerabilities | Secrets |
|-----------------------|------|-----------------|---------|
| app/package-lock.json | npm  | 0               | -       |



Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)
ubuntu@ip-172-31-46-21:~$ |
```

```
ubuntu@ip-172-31-46-21:~$ cat /var/lib/jenkins/workspace/zomato-devsecops-pipeline/trivy-image-report.txt | head -n 30

Report Summary



| Target                                                                                           | Type     | Vulnerabilities | Secrets |
|--------------------------------------------------------------------------------------------------|----------|-----------------|---------|
| pavankumarkn/zomato-devops-app:latest (alpine 3.21.3)                                            | alpine   | 17              | -       |
| opt/yarn-v1.22.22/package.json                                                                   | node-pkg | 0               | -       |
| usr/local/lib/node_modules/corepack/package.json                                                 | node-pkg | 0               | -       |
| usr/local/lib/node_modules/npm/node_modules/@isaacs/cliui/node_modules/ansi-regex/package.json   | node-pkg | 0               | -       |
| usr/local/lib/node_modules/npm/node_modules/@isaacs/cliui/node_modules/emoji-regex/package.json  | node-pkg | 0               | -       |
| usr/local/lib/node_modules/npm/node_modules/@isaacs/cliui/node_modules/string-width/package.json | node-pkg | 0               | -       |
| usr/local/lib/node_modules/npm/node_modules/@isaacs/cliui/node_modules/strip-ansi/package.json   | node-pkg | 0               | -       |
| usr/local/lib/node_modules/npm/node_modules/@isaacs/cliui/package.json                           | node-pkg | 0               | -       |
| usr/local/lib/node_modules/npm/node_modules/@isaacs/string-locale-compare/package.json           | node-pkg | 0               | -       |
| usr/local/lib/node_modules/npm/node_modules/@npmcli/agent/package.json                           | node-pkg | 0               | -       |


ubuntu@ip-172-31-46-21:~$ |
```

```
ubuntu@ip-172-31-46-21:~$ docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"
NAME          IMAGE                                     STATUS        PORTS
zomato-app     pavankumarkn/zomato-devops-app:latest   Up 46 minutes 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
sonar-server   sonarqube:lts-community                  Up 2 hours    0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp
ubuntu@ip-172-31-46-21:~$ |
```

```
ubuntu@ip-172-31-46-21:~$ echo "=== System & Tools Status ==="
node -v 2>/dev/null || /var/lib/jenkins/tools/jenkins.plugins.nodejs.tools.NodeJSInstallation/node18/bin/node -v
java -version
docker --version
trivy --version
=== System & Tools Status ===
v18.20.8
openjdk version "21.0.12" 2026-07-21
OpenJDK Runtime Environment (build 21.0.12+8-1-22.04-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.12+8-1-22.04-Ubuntu, mixed mode, sharing)
Docker version 29.1.3, build 29.1.3-0ubuntu3~22.04.2
Version: 0.74.0
ubuntu@ip-172-31-46-21:~$ |
```

THANK YOU